Listas enlazadas en Python

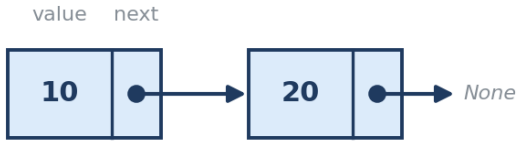
Guía visual de la clase **Linkedlist**: nodos, punteros y cada método paso a paso

1. ¿Qué es una lista enlazada?

Una lista enlazada es una secuencia de **nodos**. Cada nodo guarda un dato (**value**) y una referencia al nodo siguiente (**next**). A diferencia de una `list` de Python, los nodos no están contiguos en memoria: para llegar al nodo número *n* hay que empezar por el principio y seguir los punteros uno a uno.

En Python no existen los punteros de C; lo que hay son **referencias a objetos**. Escribir `a.next = b` no copia nada: solo hace que `next` apunte al mismo objeto que `b`. Eso es exactamente lo que dibujan las flechas de este documento.

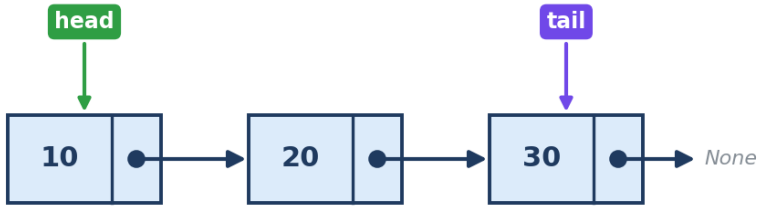
Anatomía de un nodo: value (el dato) y next (puntero al siguiente)



La clase Linkedlist guarda tres atributos que describen toda la lista:

Atributo	Qué guarda
`head`	Referencia al primer nodo (o None si la lista está vacía).
`tail`	Referencia al último nodo. Permite añadir al final sin recorrer la lista.
`length`	Número de nodos. Evita tener que recorrer la lista para contarlos.

Una lista con 3 nodos: length = 3



Cómo leer los diagramas

Azul	Nodo normal de la lista.
Naranja claro	Nodo nuevo que se está creando o insertando.
Rojo claro	Nodo que va a salir de la lista (o que ya salió).
Verde claro	Nodo encontrado por una búsqueda.
Flecha naranja gruesa	Puntero que cambia en ese paso concreto.
Etiquetas de colores	Variables que apuntan a un nodo: head, tail, temp, pre, prev, before, after.

Nota importante: en el código que me pasaste, el método `insert` tiene un fallo: falta la línea `temp.next = new_node`, así que el nodo nuevo nunca se enlaza a la lista (y `length` aumenta igualmente). En este documento aparece ya corregido y marcado en amarillo; se explica en la sección de `insert`.

2. El nodo (Node.py)

Tu clase importa `Node` desde otro archivo que no me has pasado, así que he supuesto la versión habitual: un nodo con un dato y un puntero que empieza en `None`.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

Todo nodo nuevo nace "suelto": su `next` es None hasta que algún método lo conecte con otro nodo.

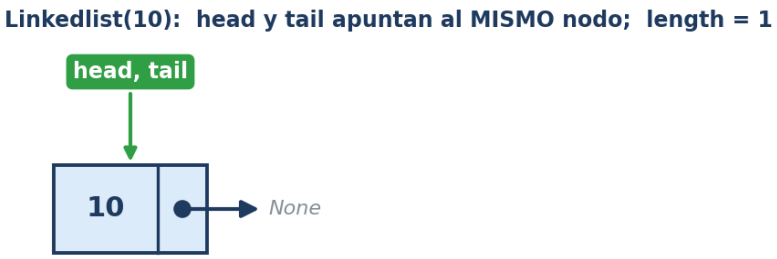
3. Los métodos, uno a uno

3.1 __init__ y print_list

El constructor crea la lista con un primer nodo. Por eso **head** y **tail** apuntan al mismo nodo y **length** vale 1.

```
    def __init__(self,value):
        new_node = Node(value)
        self.head = new_node
        self.tail = new_node
        self.length = 1
```

- Solo hay un objeto nodo; `head` y `tail` son dos nombres para el mismo objeto.
- Con este constructor no se puede crear una lista vacía directamente; una lista vacía solo aparece después de vaciarla con `pop` o `pop_first`. Por eso el resto de métodos contemplan el caso `length == 0`.




```
    def print_list(self):
        temp = self.head
        while temp is not None:
            print(temp.value)
            temp = temp.next
```

- `temp` empieza en `head` y avanza con `temp = temp.next` hasta salirse de la lista (`None`).
- Este patrón de recorrido, `while temp is not None`, es la base de casi todos los algoritmos con listas enlazadas.
- Nunca se mueve `self.head`: si lo hicieras perderías el principio de la lista.

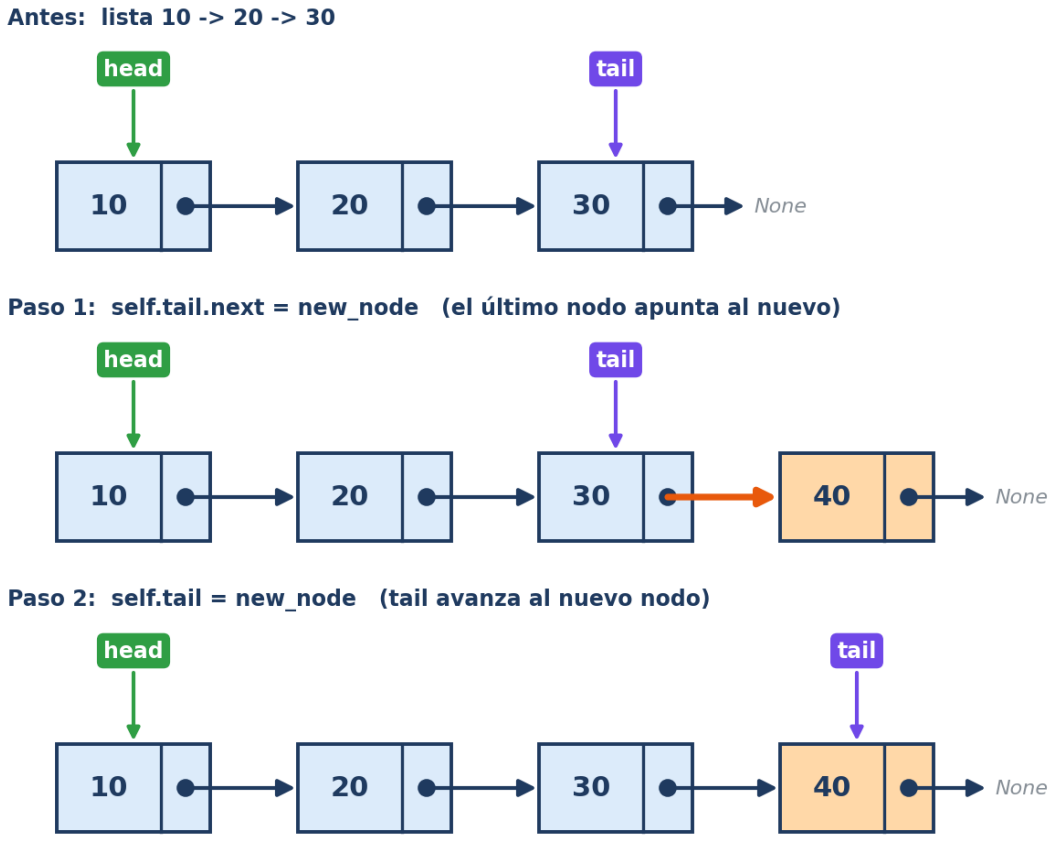
3.2 append(value) - añadir al final

Crea un nodo y lo coloca detrás del último. Gracias a **tail** no hace falta recorrer la lista.

```
    def append(self,value):
        new_node = Node(value)

        #Empty list then head and tail point to the new_node:
        if self.length == 0:
            self.head = new_node
            self.tail = new_node
        else:
            #Update the pointer first and then add the new_node:
            self.tail.next = new_node
            self.tail = new_node
        self.length += 1
        return True
```

- **Orden importante:** primero se enlaza el último nodo con el nuevo (`self.tail.next = new_node`) y solo después se mueve `tail`. Si lo hicieras al revés, perderías la referencia al último nodo antiguo.
- Caso especial: si la lista está vacía, el nodo nuevo es a la vez `head` y `tail`.
- Coste: O(1).



3.3 prepend(value) - añadir al principio

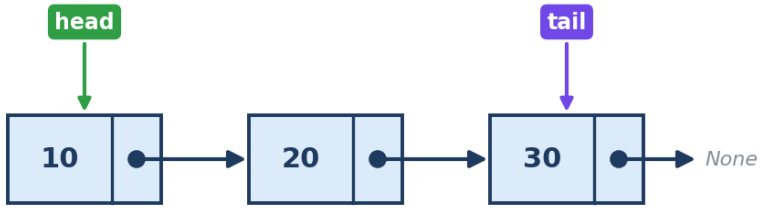
Crea un nodo y lo coloca delante del primero.

```
    def prepend(self,value):
        new_node = Node(value)

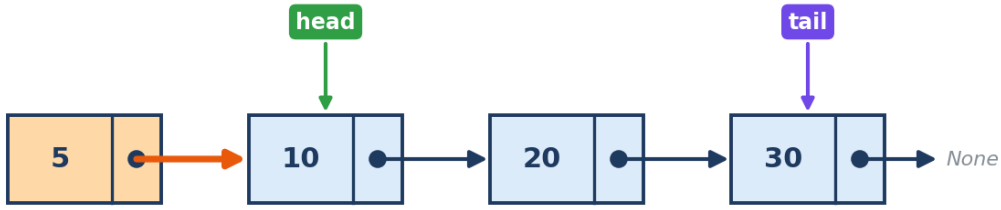
        if self.length == 0:
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head = new_node
        self.length += 1
        return True
```

- **Orden importante:** primero el nodo nuevo apunta al antiguo primer nodo (`new_node.next = self.head`) y después `head` pasa a ser el nuevo. Si movieras `head` antes, perderías toda la lista.
- Si la lista estaba vacía, `tail` también debe apuntar al nuevo nodo.
- Coste: O(1).

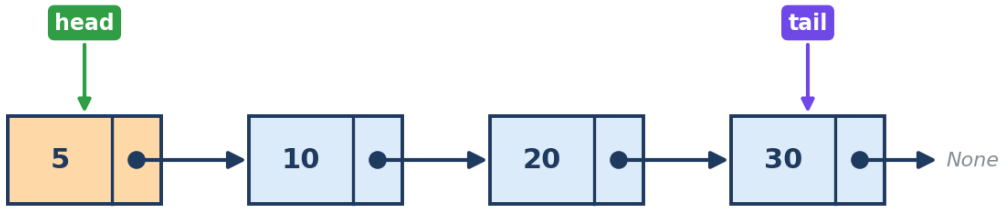
Antes: lista 10 -> 20 -> 30



Paso 1: new_node.next = self.head (el nuevo apunta al primer nodo)



Paso 2: self.head = new_node (head retrocede al nuevo nodo)



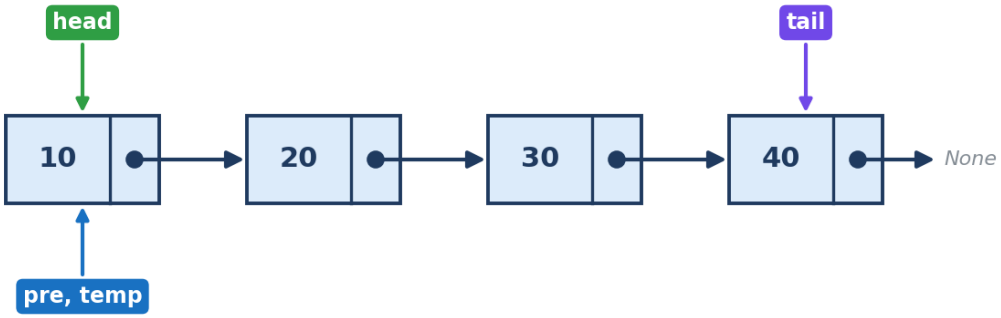
3.4 pop() - quitar el último

Elimina y devuelve el último nodo. Como la lista es simple (los nodos solo conocen al siguiente), no hay forma de ir hacia atrás: hay que recorrerla entera para encontrar el **penúltimo** nodo.

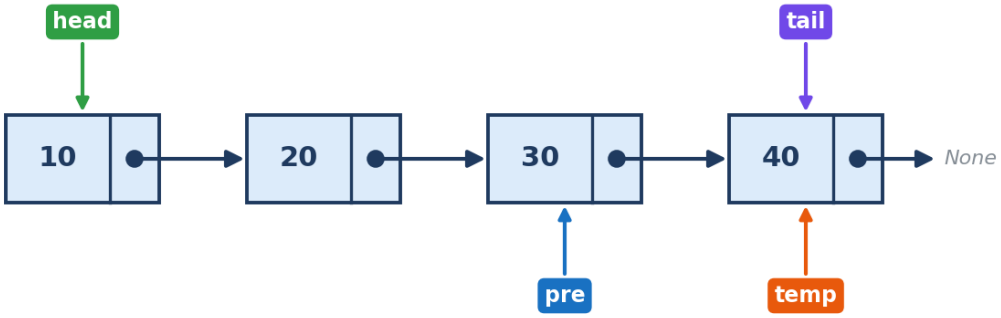
```
    def pop(self):
        if self.length == 0:
            return None
        #Iniciar variables apuntando al primer nodo. Temp irá al último nodo y pre al penúltimo
        temp = self.head
        pre = self.head
        while temp.next is not None:
            pre = temp
            temp = temp.next
        #El final ahora es el penultimo nodo y el puntero no apunta a nada
        self.tail = pre
        self.tail.next = None
        self.length -=1
        #Si teniamos un elemento y lo hemos quitado, se asocia el inicio y final a nada:
        if self.length == 0:
            self.head = None
            self.tail = None
        return temp
```

- `temp` avanza hasta el último nodo y `pre` va siempre un paso por detrás, así al terminar `pre` es el penúltimo.
- El penúltimo nodo pasa a ser el nuevo `tail` y su `next` se pone a None, con lo que el último nodo queda fuera de la lista.
- Caso especial: si solo había un nodo, `pre` y `temp` son el mismo; tras restar 1 a `length` quedan 0 elementos y por eso se ponen `head` y `tail` a None.
- Coste: O(n), porque hay que recorrer la lista.

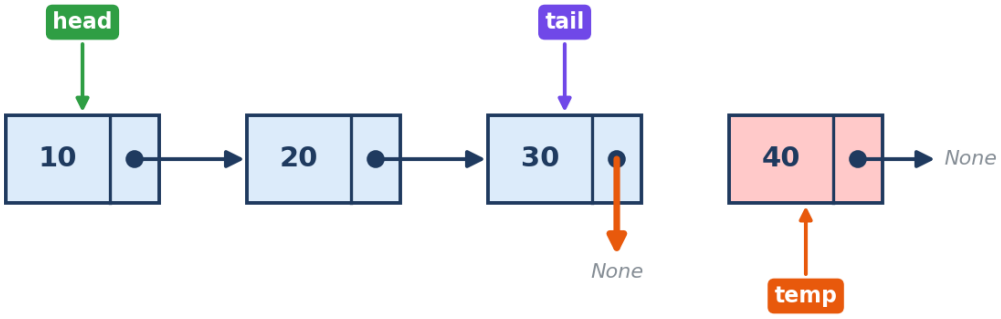
Inicio: temp = self.head ; pre = self.head



Fin del while: pre en el penúltimo nodo, temp en el último



self.tail = pre ; self.tail.next = None -> se devuelve temp (el 40)



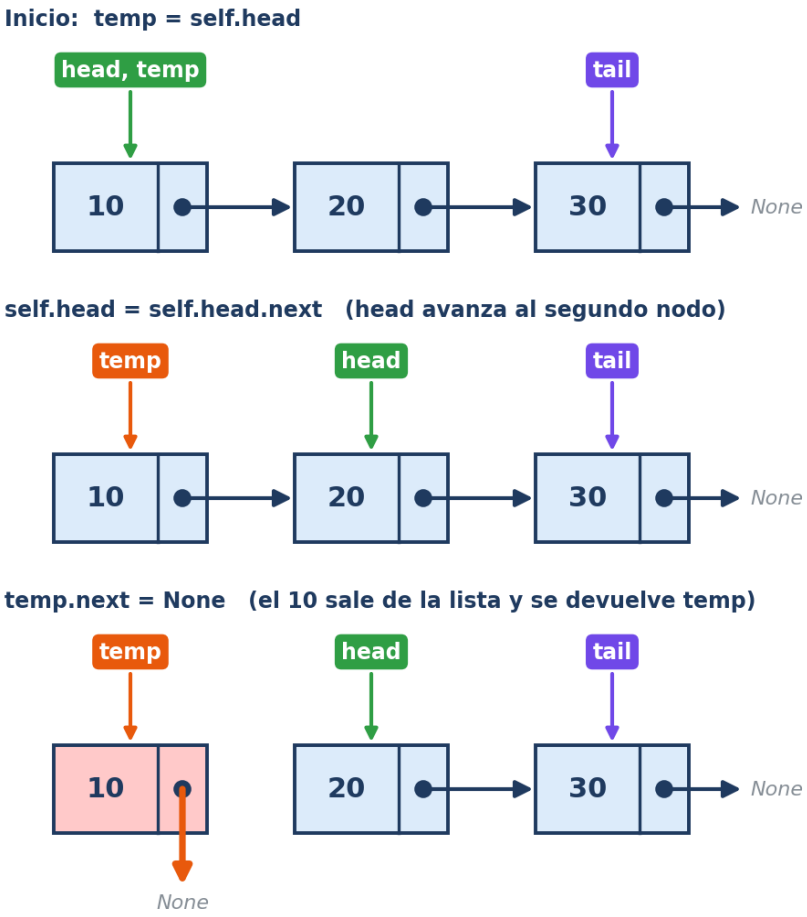
3.5 pop_first() - quitar el primero

Elimina y devuelve el primer nodo. No necesita recorrer nada.

```
    def pop_first(self):
        #Sin elementos:
        if self.length == 0:
            return None
        #1 o más elementos. Se obtiene el primer elemento y el puntero del siguiente
        temp = self.head
        self.head = self.head.next
        #Quitamos el puntero y decrementa la longitud
        temp.next = None
        self.length -= 1
        #Si habia un elemento y ahora no hay, el final apunta a None
        if self.length == 0:
            self.tail = None
        return temp
```

- Se guarda el primer nodo en `temp`, `head` avanza al segundo nodo y se corta el enlace de `temp` con `temp.next = None`.
- Si la lista queda vacía, `head` ya es None (porque venía de `head.next`) pero `tail` sigue apuntando al nodo viejo; por eso se pone a None explícitamente.

- Coste: O(1).

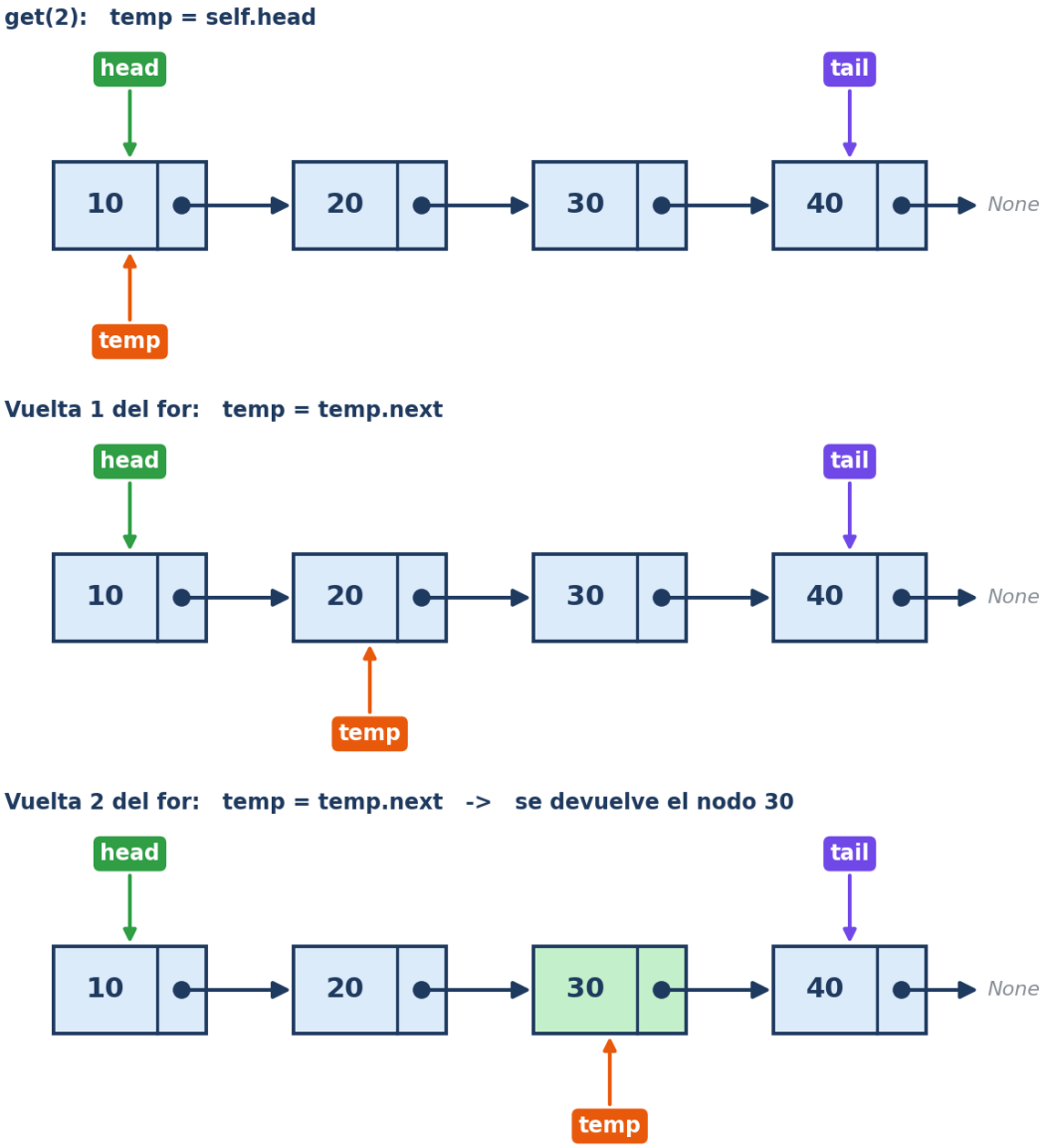


3.6 get(index) - obtener un nodo

Devuelve el **nodo** (no el valor) que está en una posición, o None si el índice no es válido.

```
#El metodo devuelve None si el indice es invalido. Devuelve el nodo si el nodo es correcto
def get(self,index):
    if index < 0 or self.length <= index:
        return None
    else:
        temp = self.head
        for _ in range(index):
            temp = temp.next
        return temp
```

- Las posiciones empiezan en 0. Se comprueba `index < 0` o `index >= length` antes de recorrer.
- El bucle `for _ in range(index)` avanza `temp` exactamente *index* veces desde `head`.
- Este método es la pieza que reutilizan `set_value`, `insert` y `remove`.
- Coste: O(n).

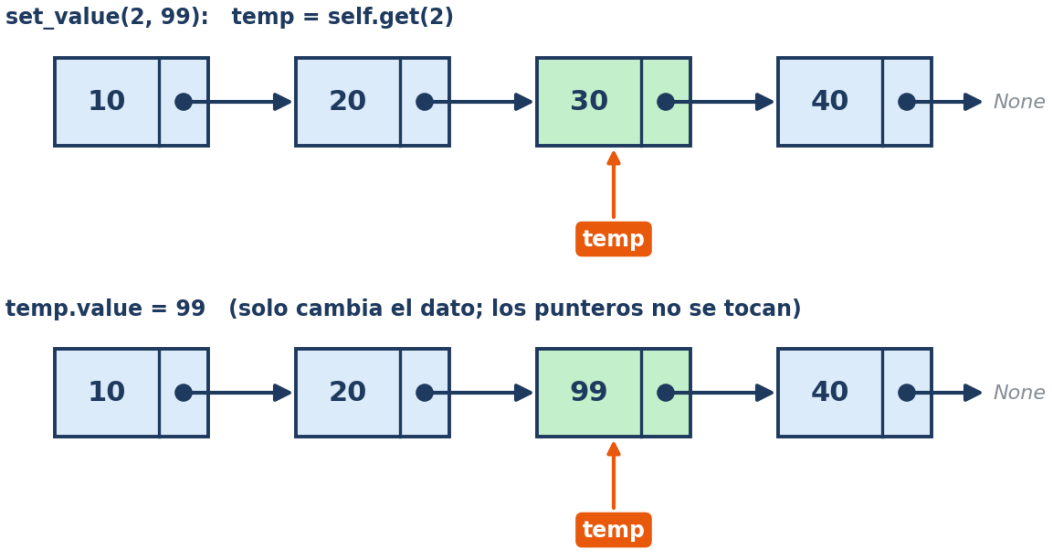


3.7 set_value(index, value) - cambiar un dato

Busca el nodo con `get` y sobrescribe su dato. Devuelve True si lo logró y False si el índice era inválido.

```
    # Se obtiene el nodo None o valido y se asigna el valor. Sobreescribe el valor del nodo.
    # Devuelve True si hizo la operacion correctamente y False si get devolvio None
    def set_value(self,index,value):
        temp = self.get(index)
        if temp:
            temp.value = value
            return True
        return False
```

- Solo cambia `value`: ningún puntero se modifica, por eso es seguro.
- `if temp:` funciona porque un nodo es "verdadero" y None es "falso".
- Coste: O(n) por culpa de `get`.



3.8 insert(index, value) - insertar en medio

Inserta un nodo en cualquier posición. Los extremos se delegan en `prepend` y `append`; el caso del medio requiere dos cambios de puntero.

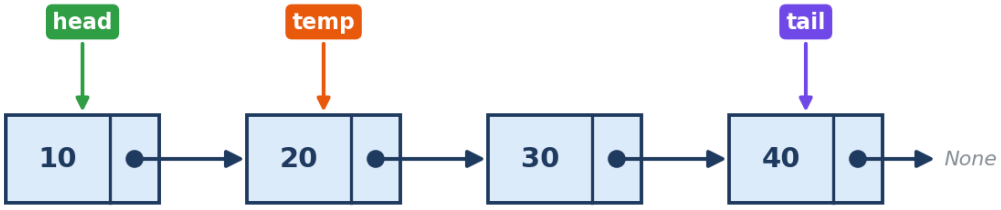
```
    def insert(self,index,value):
        #Controlar nodo invalido:
        if index < 0 or self.length < index:
            return False
        #Insertar al principio de la lista:
        if index == 0:
            return self.prepend(value)
        #Insertar al final de la lista:
        if index == self.length:
            return self.append(value)

        #Generar el nodo y apuntar a la posicion previa a la insercion para cambiar el puntero
        new_node = Node(value)
        temp = self.get(index-1)
        if temp:
            #Primero se asocia el puntero del nuevo nodo hacia el siguiente elemento de la lista antes de insertar el nodo
            new_node.next = temp.next
            #En la lista original, el elemento apunta al nuevo nodo.
            temp.next = new_node  # <-- LINEA QUE FALTABA
            self.length += 1
            return True
```

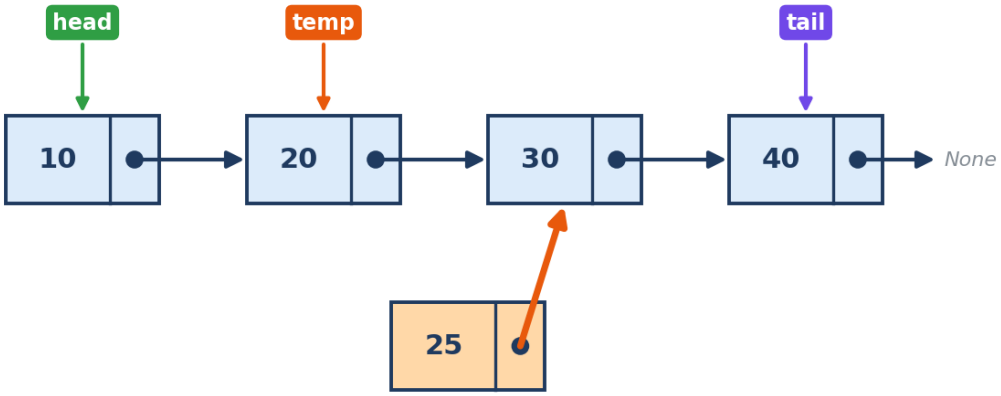
- Se permite `index == length` (insertar justo detrás del último), por eso la comprobación inicial es `self.length < index`.
- Se busca el nodo anterior a la posición con `get(index - 1)`.
- **Orden importante:** primero el nuevo apunta al siguiente (`new_node.next = temp.next`) y después el anterior apunta al nuevo (`temp.next = new_node`). Si inviertes el orden, pierdes el resto de la lista.
- Coste: O(n) en medio; O(1) en los extremos.

El fallo de tu código: sin la línea `temp.next = new_node`, el nodo 20 sigue apuntando al 30 y el nodo nuevo queda colgando, apuntando al 30 pero sin que nadie apunte a él. Al recorrer la lista el 25 no aparece, aunque `length` diga 5.

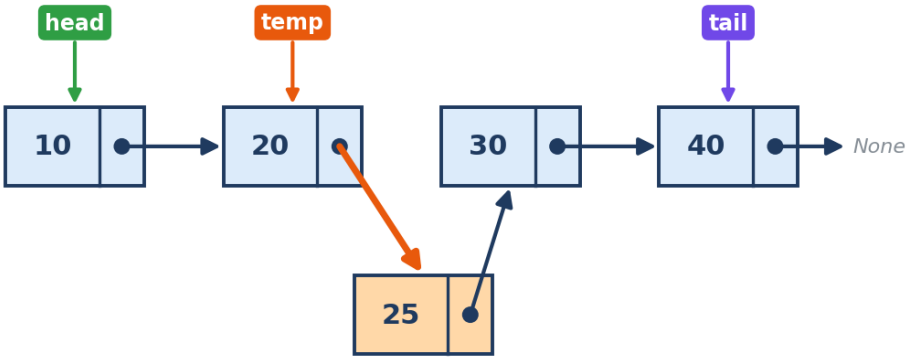
Antes: insert(2, 25) -> temp = self.get(index - 1) (temp = nodo 20)



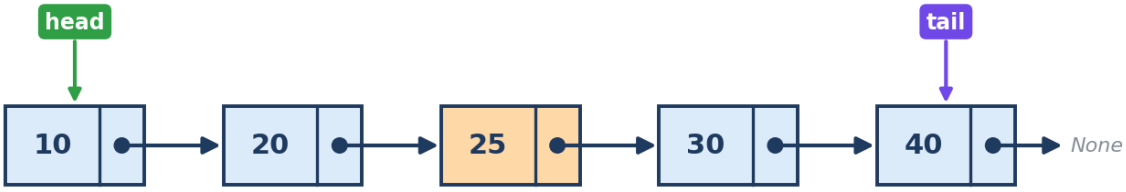
Paso 1: new_node.next = temp.next (el nuevo apunta al 30)



Paso 2: temp.next = new_node <-- esta línea faltaba en tu código



Resultado: 10 -> 20 -> 25 -> 30 -> 40 (length pasa de 4 a 5)

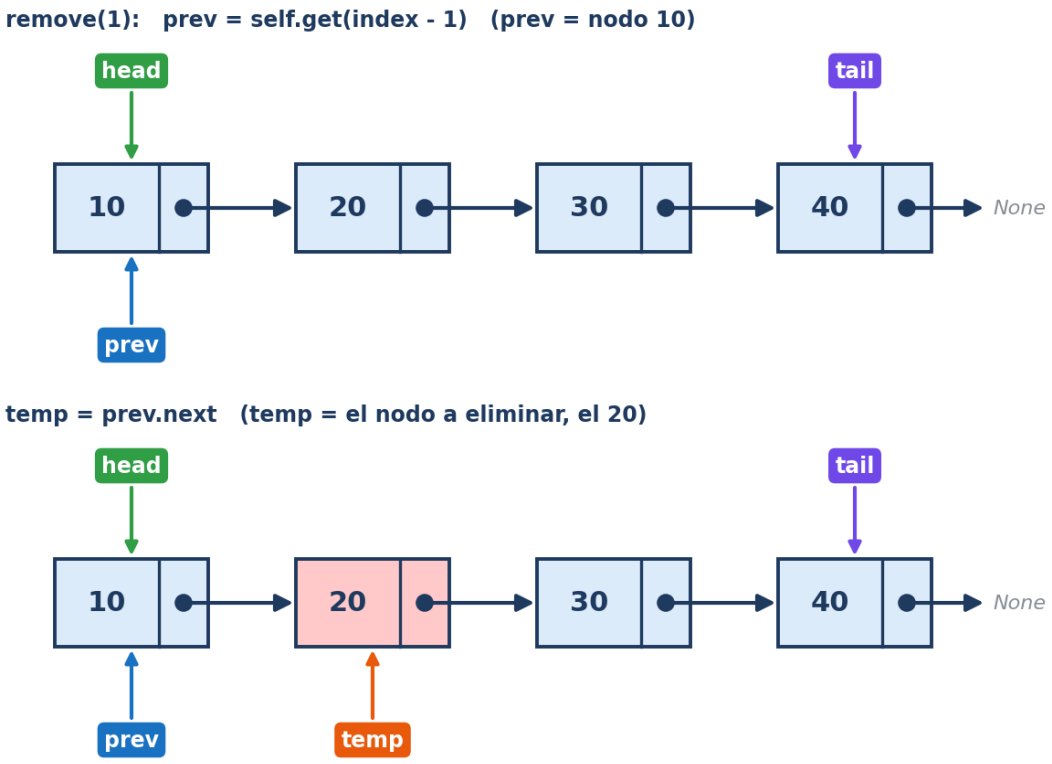


3.9 remove(index) - eliminar un nodo

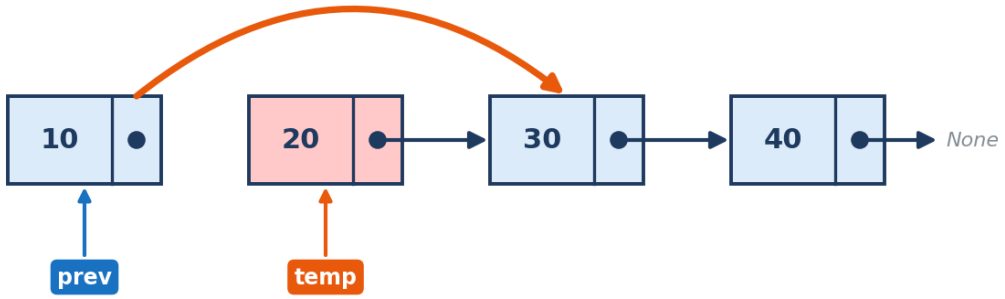
Elimina el nodo de una posición y lo devuelve. Los extremos se delegan en `pop_first` y `pop`.

```
    def remove(self,index):
        if index < 0 or self.length <= index:
            return None
        if index == 0:
            return self.pop_first()
        if index == self.length-1:
            return self.pop()
        #Elemento anterior para sobreescribir su puntero al siguiente nodo
        prev = self.get(index - 1)
        temp = prev.next
        #Movemos el puntero del previo al siguiente:
        prev.next = temp.next
        #Deshabilitamos la conexion entre el puntero del elemento a borrar y el siguiente:
        temp.next = None
        self.length -= 1
        return temp
```

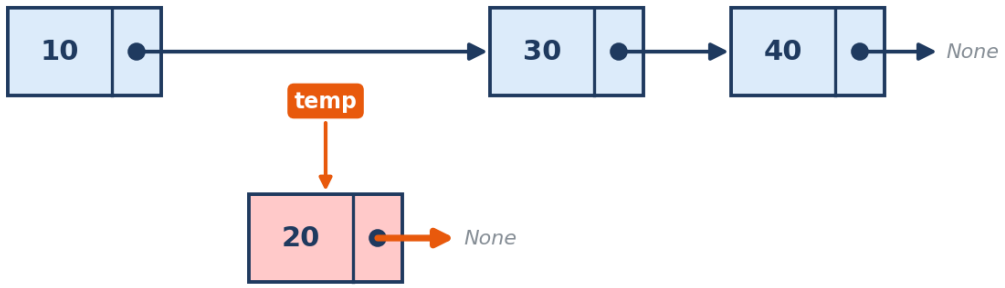
- Se necesita el nodo **anterior** (`prev`) para poder saltarse el que se elimina: `prev.next = temp.next`.
- Después se aísla el nodo con `temp.next = None`, para que no siga enganchado a la lista.
- Coste: O(n) en medio; O(1) si es el primero.



prev.next = temp.next (el 10 salta el 20 y apunta al 30)



temp.next = None (el 20 queda aislado y se devuelve temp)



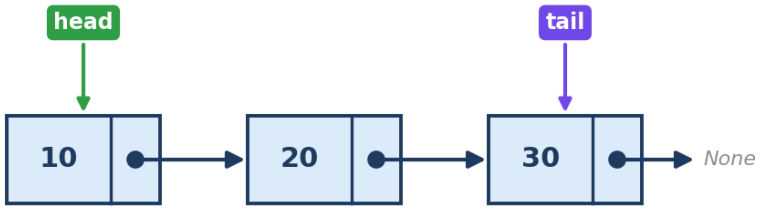
3.10 reverse() - invertir la lista

Invierte la dirección de todos los punteros sin crear nodos nuevos. Es el ejercicio clásico de listas enlazadas: se necesitan **tres** referencias que avanzan a la vez.

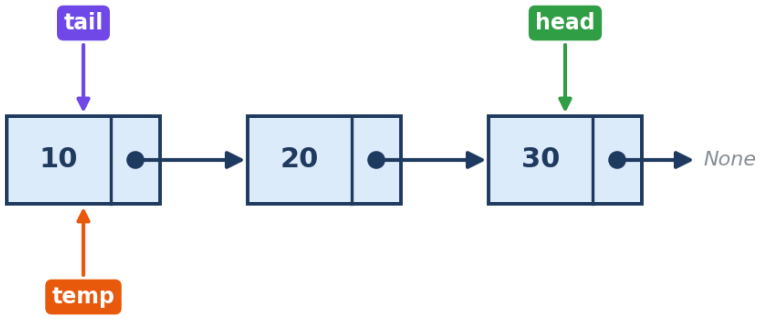
```
    def reverse(self):
        # Se obtiene el primer nodo temporal:
        temp = self.head
        # Intercambiar inicio y fin:
        self.head = self.tail
        self.tail = temp
        # Generar el puntero posterior a cada nodo (el anterior no por si la lista tiene 0 elementos):
        before = None
        # Iteracion a traves de la lista para cambiar la direccion de los punteros
        for _ in range(self.length):
            after = temp.next  # 1) guardo el siguiente
            temp.next = before  # 2) invierto el puntero
            before = temp  # 3) avanzo before
            temp = after  # 4) avanzo temp
        return True
```

- Primero se intercambian `head` y `tail` (el antiguo primero será el último).
- En cada vuelta del `for` se hacen siempre los mismos 4 pasos, y su orden no se puede cambiar: **1)** guardar el siguiente en `after`, **2)** invertir el puntero (`temp.next = before`), **3)** avanzar `before`, **4)** avanzar `temp`.
- Si no guardaras `after` antes del paso 2, al invertir el puntero perderías el resto de la lista.
- Coste: O(n).

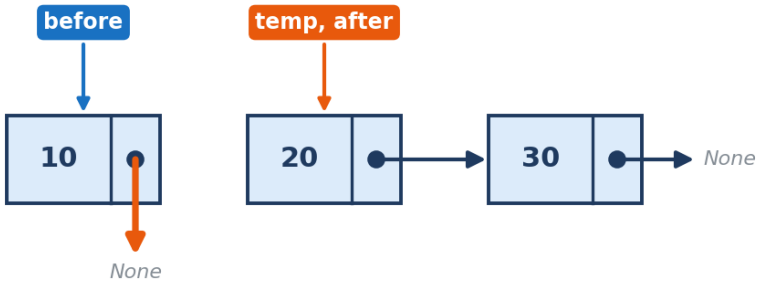
Estado inicial



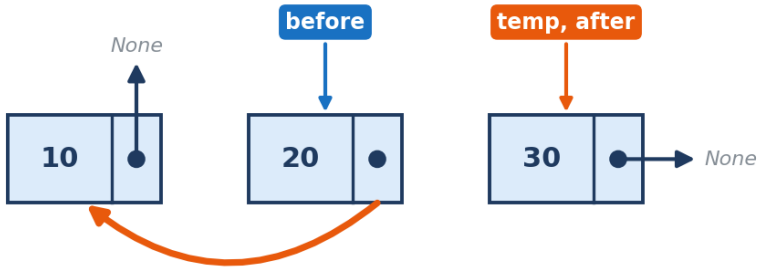
1) head y tail se intercambian; temp = antiguo head; before = None



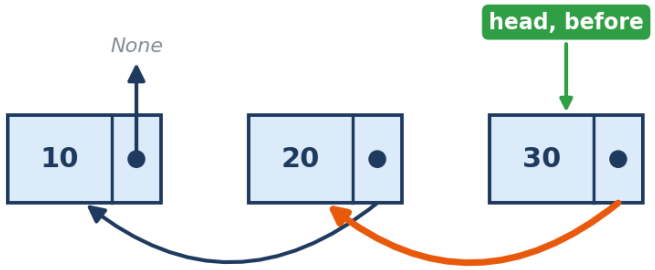
Vuelta 1: el 10 pasa a apuntar a None; before y temp avanzan



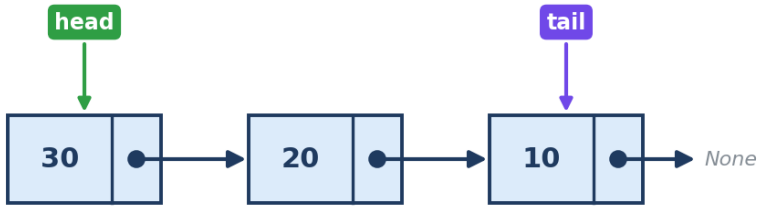
Vuelta 2: el 20 pasa a apuntar al 10



Vuelta 3: el 30 pasa a apuntar al 20; temp = None y el for termina



Resultado: la lista queda invertida



4. Resumen de costes

Método	Coste	Por qué
`append`	O(1)	Usa tail, no recorre.
`prepend`	O(1)	Solo toca head.
`pop_first`	O(1)	Solo toca head.
`pop`	O(n)	Hay que llegar al penúltimo nodo.
`get / set_value`	O(n)	Hay que recorrer desde head.
`insert`	O(n)	Recorre hasta el nodo anterior (O(1) en los extremos).
`remove`	O(n)	Recorre hasta el nodo anterior (O(1) si es el primero o el último).
`reverse`	O(n)	Una pasada por toda la lista.

Nota: el último nodo de `remove` se delega en `pop`, que es O(n) en una lista simple.

5. Ejemplo de uso con su salida real

Este programa usa la clase (con `insert` corregido). La salida es la que produce realmente al ejecutarlo; cada valor de `print_list` sale en una línea.

```
from linkedlist_src import Linkedlist

mi_lista = Linkedlist(10)
mi_lista.append(20)
mi_lista.append(30)
mi_lista.prepend(5)
print("length:", mi_lista.length)
mi_lista.print_list()

mi_lista.insert(2, 15)
print("-- tras insert(2, 15) --")
mi_lista.print_list()

mi_lista.remove(1)
print("-- tras remove(1) --")
mi_lista.print_list()

mi_lista.reverse()
print("-- tras reverse() --")
mi_lista.print_list()
```

Salida:

```
length: 4
5
10
20
30
-- tras insert(2, 15) --
5
10
15
20
30
-- tras remove(1) --
5
15
20
30
-- tras reverse() --
30
20
15
5
```

Apéndice: código completo (con insert corregido)

La única línea añadida respecto a tu versión está resaltada en amarillo.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

```
from Node import Node

class Linkedlist:
```

```
    def __init__(self,value):
        new_node = Node(value)
        self.head = new_node
        self.tail = new_node
        self.length = 1
```

```
    def print_list(self):
        temp = self.head
        while temp is not None:
            print(temp.value)
            temp = temp.next
```

```
    def append(self,value):
        new_node = Node(value)

        #Empty list then head and tail point to the new_node:
        if self.length == 0:
            self.head = new_node
            self.tail = new_node
        else:
            #Update the pointer first and then add the new_node:
            self.tail.next = new_node
            self.tail = new_node
        self.length += 1
        return True
```

```
    def pop(self):
        if self.length == 0:
            return None
        #Iniciar variables apuntando al primer nodo. Temp irá al último nodo y pre al penúltimo
        temp = self.head
        pre = self.head
        while temp.next is not None:
            pre = temp
            temp = temp.next
        #El final ahora es el penultimo nodo y el puntero no apunta a nada
        self.tail = pre
        self.tail.next = None
        self.length -=1
        #Si teniamos un elemento y lo hemos quitado, se asocia el inicio y final a nada:
        if self.length == 0:
            self.head = None
            self.tail = None
        return temp
```

```
    def prepend(self,value):
        new_node = Node(value)

        if self.length == 0:
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head = new_node
        self.length += 1
        return True
```

```
    def pop_first(self):
        #Sin elementos:
        if self.length == 0:
            return None
        #1 o más elementos. Se obtiene el primer elemento y el puntero del siguiente
        temp = self.head
        self.head = self.head.next
        #Quitamos el puntero y decrementa la longitud
        temp.next = None
        self.length -= 1
        #Si habia un elemento y ahora no hay, el final apunta a None
        if self.length == 0:
            self.tail = None
        return temp
```

```
    #El metodo devuelve None si el indice es invalido. Devuelve el nodo si el nodo es correcto
    def get(self,index):
        if index < 0 or self.length <= index:
            return None
        else:
            temp = self.head
            for _ in range(index):
                temp = temp.next
            return temp
```

```
    # Se obtiene el nodo None o valido y se asigna el valor. Sobreescribe el valor del nodo.
    # Devuelve True si hizo la operacion correctamente y False si get devolvio None
    def set_value(self,index,value):
        temp = self.get(index)
        if temp:
            temp.value = value
            return True
        return False
```

```
    def insert(self,index,value):
        #Controlar nodo invalido:
        if index < 0 or self.length < index:
            return False
        #Insertar al principio de la lista:
        if index == 0:
            return self.prepend(value)
        #Insertar al final de la lista:
        if index == self.length:
            return self.append(value)

        #Generar el nodo y apuntar a la posicion previa a la insercion para cambiar el puntero
        new_node = Node(value)
        temp = self.get(index-1)
        if temp:
            #Primero se asocia el puntero del nuevo nodo hacia el siguiente elemento de la lista antes de insertar el nodo
            new_node.next = temp.next
            #En la lista original, el elemento apunta al nuevo nodo.
            temp.next = new_node  # <-- LINEA QUE FALTABA
            self.length += 1
            return True
```

```
    def remove(self,index):
        if index < 0 or self.length <= index:
            return None
        if index == 0:
            return self.pop_first()
        if index == self.length-1:
            return self.pop()
        #Elemento anterior para sobreescribir su puntero al siguiente nodo
        prev = self.get(index - 1)
        temp = prev.next
        #Movemos el puntero del previo al siguiente:
        prev.next = temp.next
        #Deshabilitamos la conexion entre el puntero del elemento a borrar y el siguiente:
        temp.next = None
        self.length -= 1
        return temp
```

```
    def reverse(self):
        # Se obtiene el primer nodo temporal:
        temp = self.head
        # Intercambiar inicio y fin:
        self.head = self.tail
        self.tail = temp
        # Generar el puntero posterior a cada nodo (el anterior no por si la lista tiene 0 elementos):
        before = None
        # Iteracion a traves de la lista para cambiar la direccion de los punteros
        for _ in range(self.length):
            after = temp.next  # 1) guardo el siguiente
            temp.next = before  # 2) invierto el puntero
            before = temp  # 3) avanzo before
            temp = after  # 4) avanzo temp
        return True
```